LUVANI

Backend production readiness audit

Architecture, security, voice reliability and launch operations

PUBLIC LAUNCH: HOLD

30 prioritized findings. Reproduced availability and admission defects require remediation before public exposure. Critical deployment and voice acceptance evidence remains unavailable.

3 October 2026 | Europe/London

Source baseline: 2cbea8655b90c22fcac00412d008d4540e69954d

Prepared for the founder, backend engineers and release owner.

Includes the current-state audit, required changes with acceptance checks, proposed production architecture, four specialist agent definitions, a 25-practice industry synthesis and the 100-app/service source register.

Evidence is explicitly scoped. Local tests do not certify a deployed service, and public disclosures do not expose complete private app architectures.

Contents

What the evidence establishes 4

Verification snapshot . . . 4

Direct answers to the requested questions 5

How the backend currently works 6

Assessment against the supplied infrastructure layers 7

Prioritized findings and required changes 7

A01 P0 Unauthenticated malformed Host terminates the API . . . 7

A02 P1 Oversized scoring audio also terminates the API . . . 8

A03 P1 Voice replay defense is process-local in a process-per-job runtime . . . 8

A04 P1 The abandon endpoint defeats the active-session spend boundary . . . 8

A05 P1 Shutdown and delivery retries do not guarantee settlement or saved practice . . . 8

A06 P1 Main conversation brain failures can remain billable until the session ends . . . 9

A07 P1 Safety scans a prefix while the model receives the full input . . . 9

A08 P1 Shared TTS cache retains personalized speech indefinitely . . . 9

A09 P1 Direct Supabase writes bypass client and API resource bounds . . . 9

A10 P1 Account export fails above fixed size and row ceilings . . . 9

A11 P1 Account deletion depends on a billing processor even when billing is dark . . . 10

A12 P1 Published retention promises lack deployed enforcement . . . 10

A13 P1 Intended migrations and actual deployment have no verified compatibility contract . . . 10

A14 P1 The hosted database verification harness is stale . . . 10

A15 P1 Backup claims and fallback coverage are unreliable . . . 11

A16 P1 Public hosting and native release configuration are not demonstrated . . . 11

A17 P1 CI succeeds but does not enforce the release boundary . . . 11

A18 P1 Observability cannot establish service quality or detect money/privacy failures . . . 11

A19 P1 Voice and learning quality still require real release acceptance . . . 11

A20 P1 Whole-product vendor spend and admission need enforceable bounds . . . 12

A21 P2 JWT verification needs an explicit token contract and revocation policy . . . 12
A22 P2 Strict command schemas and bounded queues are incomplete . . . 12
A23 P2 Raw error logging can disclose learner-derived content . . . 12
A24 P2 A hung sync request blocks subsequent account reconciliation . . . 12
A25 P2 Query pagination and aggregate cost need realistic-volume proof . . . 13
A26 P2 Friends is self-reported and moderation is not operational . . . 13
A27 P2 Durable practice recovery needs consistent deadlines and operational deployment . . . 13
A28 P2 Configuration and failure semantics need consistent production guards . . . 13
A29 P2 Dependency acceptance and history scanning need stronger freshness guarantees . . . 14
A30 P2 Smaller integrity and cost-accounting edges remain . . . 14

Proposed production architecture **14**

Efficiency and quality measurements to implement **15**

Implementation order and release exit conditions **15**

Backend architecture agents added for future changes and releases **16**

Industry benchmark and limits **17**

Reproducible evidence package **17**

Remaining verification and access limits **17**

Industry practices and source comparison **18**

Practices that should change the launch standard **18**

Architectural guidance **20**

Evidence limits and open questions **20**

Mapping the benchmark to this audit **21**

100-app/service source register **22**

Audit date: 3 October 2026, Europe/London. **Source baseline:** 2cbea8655b90c22fcac00412d008d4540e69954d. **Decision:** HOLD public launch. **Audience:** founder, backend engineers and release owner.

Luvani has a credible modular backend and meaningful security controls. It is not ready for unrestricted public production on the evidence available. The most urgent defects are reproducible token-server process termination, ineffective replay protection across actual voice jobs, a client-callable escape from the session concurrency lease, unreliable finalization, incomplete data-resource limits and unexpected persistence of personalized tutor audio. The configured Supabase project is inactive, the latest hosted schema cannot be verified, and production hosting, recovery and real-device voice acceptance remain unproved.

This report recommends changes; it does not implement the application fixes. The separately requested four backend architecture agent definitions and their review routing have been added. No deployment, hosted migration, project restore, account change, key rotation or scheduled automation was performed.

What the evidence establishes

The audit inspected API, authentication, entitlement/billing, SQL policies/functions, account erasure/export, learner sync, social features, voice workers, protocol, caching, practice recovery, CI and release configuration. It ran the existing local gate, isolated synthetic API/voice/SQL reproductions, and read GitHub/Supabase metadata. No real learner data or paid speech/model calls were used. The supplied image supplied the layer checklist; it was not treated as evidence of Luvani's implementation or as instructions.

Evidence labels used below:

- **Reproduced:** an executed isolated check demonstrates the stated behavior.
- **Source verified:** current source or installed dependency implements the stated behavior; a live end-to-end exploit was not run.
- **Live metadata:** a connected service reported the state during this audit.
- **Historical:** a repository run record, not a new measurement.
- **Unverified:** required deployed, provider, concurrency or device evidence was unavailable.

Priority is a recommendation for this report, independent of the repository's parsed issue ledger: **P0** must be resolved before network exposure; **P1** must be resolved or the affected feature kept out of public launch; **P2** is important hardening or a conditional product/capacity concern. An unknown critical production control can block release even when no unit test fails. No assertion that the app is certified secure, that every possible defect was found, or that private architectures of major apps were inspected is made.

Verification snapshot

Check	Result	Practical limit
Existing local full gate	4,232 tests pass, workspace typechecks pass	Includes substantial mock and source-contract coverage; not a public deployment test
Workspace counts	core 711; brain 464; tutor 538; API 298; playground 26; mobile 2,195	Focused runs below overlap these totals; do not add them together
GitHub CI for baseline commit	Linux gate, dependency audit and secret scan all successful	Run 37149648954; not an application deployment
GitHub main branch	protected=false; required checks enforcement off	Rulesets query returned 403 with plan-feature restriction; no effective ruleset attestation
Current npm audit	0 critical, 21 high, 12 moderate affected-package rows	Five advisory roots; inherited rows are not 33 independent exploits

Check	Result	Practical limit
Repository audit policy	Passes with current registry access	Accepts high roots by package name; newly added advisories for an accepted package are not individually reviewed by the gate
Reachable-history secret scan	Failed with child-process output buffer exhaustion	Cannot claim history clean; tracked-file scan passes. Error construction can include raw diff output
Full SQL replay	All 26 migrations apply; 21 of 21 resulting public tables have RLS	Disposable PGlite and explicit simulated Supabase grants; not a hosted concurrency or gateway proof
Seven existing SQL proof scripts	Pass	Social, billing, refund, practice queue, saved text, product events and phrase-audio budgets; sequential local evidence
Voice focused checks	538 tutor tests; 69 focused mobile checks; 7 practice checks pass	No real beginner speech, audible latency or physical-device acceptance
Configured Supabase	Canonical API/mobile configuration maps to Parle; metadata says INACTIVE	Migration-list and read-only catalog queries time out. Empty advisor output is not a healthy live-schema attestation
Canonical local environment	Mobile API hostname is localhost; LiveKit hostname is Cloud	Environment names/hostnames only inspected; credentials not emitted. Hosted release environment could differ

Existing documents contain contradictory historical statements. In particular, CI has run successfully and earlier exposed vendor keys were recorded as retired on 3 October with founder confirmation and replacement authentication checks. This report does not repeat the old claim that those keys remain live. Current full-history scanning still needs repair and re-verification.

Direct answers to the requested questions

Does the backend complement the app? Mostly at the design level. The shared pedagogy, typed twins, server-owned minutes, local-first progress and conversation-derived practice fit the product. Execution gaps undermine that fit: lost post-session work, stalled sync, billing during definitive brain failure, conditional recovery flags and unavailable live pronunciation evidence.

Is it efficient? Some important optimizations exist: prompt-prefix caching, authored scenes, voice cache fingerprints, prewarmed VAD, disabled speculative conversation generation and bounded vendor deadlines. Efficiency in public use is unproved. There is no measured launch capacity, enforceable whole-product spend ceiling, bounded command queue, complete query pagination or percentile telemetry. The current general speech cache saves money by retaining data it should not retain indefinitely.

Are the voice tutors working? They are real implementations, not frontend simulations. Unit tests and historical local LiveKit lifecycle runs establish useful behavior. They do not establish that the deployed public service is currently serving, that native users hear timely replies, or that beginner French is reliably recognized. Live pronunciation scoring is deliberately default-off; free conversation reports no pronunciation score. Dedicated scoring drills are a separate path.

Is version control in place? Yes: Git, remote history, lockfile, migrations, hooks and a working CI pipeline exist. Release enforcement is weaker: main is unprotected, there is no checked-in CD/service deployment contract or verified artifact promotion, and the recorded 20 September hosted migration versions differ from local filenames. Current hosted migration history is unverified. Code version control is not the same as infrastructure and release versioning.

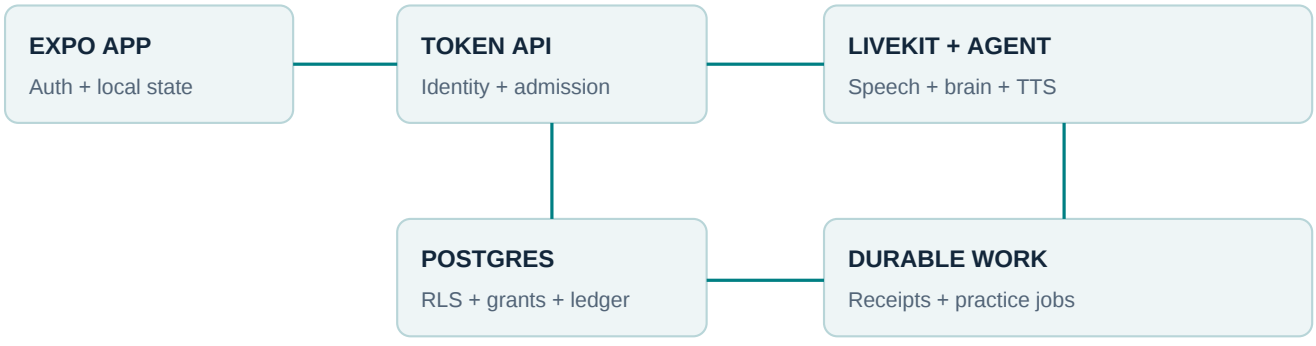
Is RLS in check? The complete intended local schema enables RLS on every public table and meaningful role/owner tests pass. Privileged RPC grants are restricted. Current deployed RLS cannot be attested while the configured database is inactive. RLS also does not stop excessive writes to one's own records, validate client learning claims, or authorize calls made with a service key.

Is top-level security implemented? Several foundations are implemented; the full production security envelope is incomplete. Uncaught API failures, voice admission, direct Data API quotas, JWT claim/revocation policy, cache/log privacy, auth anti-abuse settings, secret governance, monitoring and recovery all need work or deployed verification. Use OWASP ASVS 5.0.0 and the API resource-consumption guidance as acceptance frameworks, not as a certification claim.

Is it built like a leading app backend? The core service boundaries are appropriate for an early production product. It lacks the operational guarantees that distinguish a mature backend: enforceable admission, durable work, measured service quality, controlled releases, abuse economics, privacy lifecycle and exercised recovery. Adding Kubernetes, many microservices or database sharding would not resolve the current problems.

How the backend currently works

1. The Expo app authenticates through Supabase and stores the native session through a secure-storage adapter. Learner records live locally under account-scoped storage; one sync engine merges namespace envelopes with authenticated PostgREST rows and database revisions.
2. The app asks the Node token server to start a lesson or conversation. The server verifies a bearer, checks IP/user/installation budgets, claims introductory time, reads entitlements, signs a LiveKit credential, acquires a Redis account lease and records a durable session grant before returning the credential.
3. A LiveKit worker admits a job using signed metadata. Each real SDK job executes in a fresh child process. Speech uses VAD/STT, a lesson or conversation engine, and selected TTS. Typed twins use the same progression without speech clients. Core/brain owns learning decisions; the client renders messages.
4. The agent computes duration and sends authenticated end-of-session settlement. The database fences ownership, grant epoch, charged delta and replay, and can refund recognized terminal service failures. The server releases the account lease.
5. Conversation finishing prepares a usable baseline and later enrichment. If its baseline reaches the API with durable-practice mode enabled, SQL atomically receives it and creates a recovery job. A separate polling worker claims leased jobs and regenerates drills. This queue cannot recover a baseline never received.
6. RevenueCat webhooks and learner refresh read vendor subscriber state. Database-issued refresh ordering and a row lock apply the current projection and event completion atomically. Purchases deliberately stay closed without complete configuration and schema readiness.
7. Friends uses an authenticated invoker wrapper and private privileged dispatcher, with opt-in, mutual relationships and blocks. Its points are capped aggregates of client-authored activity, explicitly disclosed as self-reported rather than anti-cheat-certified ability.



Redis limits/leases and RevenueCat reconciliation support the control path.

Assessment against the supplied infrastructure layers

Layer	Current position	Required improvement
Frontend integration	Shared protocol, secure session storage, local-first continuity and typed alternatives	Actual native/build/network acceptance; explicit feature availability and client-version compatibility
APIs and backend logic	Central identity/minutes boundary; owner filters and bounded vendor adapters	Contain all async failures; strict input schemas; versioned contracts; readiness; pagination
Database and storage	Ordered SQL, RLS, atomic billing and queue fencing	Running release schema; quotas; retention; catalog manifest; complete backup/export and query evidence
Auth and permissions	Asymmetric verification, owner policies and recent-auth checks	Explicit token claims/revocation; verified auth-provider/CAPTCHA/email settings; least-privilege operator access
Hosting and deployment	Fail-closed boot and Cloud-compatible code	Reproducible service artifacts, production/staging isolation, managed secrets, drain/canary/rollback evidence
Cloud and compute	Hosted vendor endpoints configured locally	Capacity/admission budget, region placement, finite queues and resource limits; verified running fleet
CI/CD and version control	Working Linux CI, Git, lockfile and migration files	Required PR checks, protected main, artifact/schema promotion and immutable release identity
Security and RLS	Locally meaningful isolation and restricted privileged RPCs	Close reproduced availability/admission/privacy holes; audit direct routes and actual hosted privileges
Rate limiting	Distributed API limits and leases	Direct Supabase quotas, voice-command limits, global vendor concurrency/spend, correct proxy trust
Caching and CDN	Prompt/audio caching and bundled static content	Static allowlist caching, retention/eviction, cache-hit measurement, hosted content cache/version policy
Load balancing and scaling	Managed media path and shared state primitives	Correct admission across jobs first, then measured capacity, readiness-based routing and graceful drain
Errors and logs	Usage/timing and controlled product events exist	Redacted structured logs, traces, error budgets, dashboards and actionable alerts
Availability and recovery	Local retries, SQL rollback scripts and recovery queue	Durable settlement/outbox, backup coverage, tested restore, on-call/incident process and failure drills

Prioritized findings and required changes

A01 P0 Unauthenticated malformed Host terminates the API

Reproduced. services/token-server/src/server.ts:252 constructs a URL from the caller's Host header before entering the request try/catch. A synthetic raw request with an invalid Host reached the isolated API, produced no HTTP response and exited Node with code 1 and ERR_INVALID_URL. No bearer or vendor call was involved. A specific edge proxy may reject this input; no such enforced deployment boundary was verified.

Parse relative request paths against a fixed internal base inside the protected handler, reject malformed requests with 400 and attach a rejection boundary to every async HTTP listener. Do not rely on an unhandled-rejection process policy to keep serving. **Acceptance:** raw malformed authority/path/header cases return bounded errors; the same PID serves the next health and authenticated request; test both direct API and the selected edge proxy. **Owners:** API and security engineers.

A02 P1 Oversized scoring audio also terminates the API

Reproduced. drillScore.ts:63 throws above 2 MiB. server.ts:485 returns the async handler without awaiting it, so its rejected promise escapes the outer catch. Sending 2 MiB plus one byte with a permitted synthetic local identity disconnected the client and exited Node with code 1. Authentication was intentionally mocked by explicit development mode; this proves exception handling, not a production auth bypass. The same overflow is reachable after real authentication.

Await route handlers within one error boundary; handle oversize/invalid/interrupted bodies explicitly and cancel work on client disconnect. Audit all delegated routes for the same promise pattern. **Acceptance:** overflow returns 413, malformed input 400, dependency timeout 503, and aborted uploads never terminate the API or retain unbounded buffers. **Owners:** API/security engineers.

A03 P1 Voice replay defense is process-local in a process-per-job runtime

Source verified and independently reproduced at the process boundary. tutor.ts:101 keeps reported IDs in a Set; agent.ts:473 and installed LiveKit 1.9.0 job-process code establish fresh one-shot child processes. Marking a session in one process leaves it unreported in another. The entry path has no atomic shared admission check. Debit idempotency stops a repeated charge, not additional vendor work.

Implement service-owned atomic grant admission bound to account, room, allowance epoch and a job fencing token. Distinguish reconnecting an admitted job from dispatching a new job, and refuse ended/revoked/abandoned grants before speech/model work. **Acceptance:** race/replay/restart tests across real SDK jobs and replicas dispatch zero new provider work for a spent grant while valid reconnects settle once. A paid live replay was not executed. **Owners:** voice, API, security and revenue engineers.

A04 P1 The abandon endpoint defeats the active-session spend boundary

Reproduced API behavior; simultaneous live-room cost is inferred. server.ts:786-812 lets the authenticated owner release a matching lease without checking or terminating an active room. The isolated sequence was start 200, second start 409, abandon 200, new start 200. No token revocation or room termination is performed at this code boundary; live rejoin with the first signed token was not tested. Starts read the current allowance before prior rooms settle. At ten user session requests per minute, with starts and abandons sharing that counter, abuse is bounded by request rate but can still accumulate live rooms.

The risk is vendor spend and fleet occupancy, not only the user making their own concurrency worse. Make abandon an authoritative state transition that prevents subsequent admission of the old credential and confirms old-job termination before replacing an active room. Add allowance reservations/global admission rather than relying solely on after-session debit. LiveKit documents that token expiry controls initial connection rather than an ongoing session; Cloud refreshed-token revocation cutoff and self-hosted removal behavior need explicit acceptance. **Acceptance:** malicious abandon/restart loops and reconnects never exceed the intended active-job/issued-time allowance, including races, disconnected agents and a Redis outage. **Owners:** voice/API/revenue/security engineers.

A05 P1 Shutdown and delivery retries do not guarantee settlement or saved practice

Source verified. agent.ts:436 discards the finish promise and no application session-end/shutdown callback links it to settlement. The installed SDK closes its primary AgentSession and awaits its own lifecycle work, but does not await Luvani's dropped finish promise. usage.ts and conversationPacks.ts try three deliveries and then log. There is no durable usage/receipt outbox or abandoned-session reconciler. Durable practice begins only after successful server receipt.

Register one idempotent awaited finalization promise, align host drain grace with its budget, and persist authoritative admission/usage/finish state plus a retryable baseline receipt. Reconcile crash/uncertain acknowledgements on restart. **Acceptance:** SIGTERM, process kill, network loss and crash around debit/baseline ACK leave exactly-once settlement and recoverable practice or an honest pending state; deleted content cannot return. **Owners:** backend/voice/data/revenue/deploy engineers.

A06 P1 Main conversation brain failures can remain billable until the session ends

Reproduced with the real conversation class and mocked provider. Three definitive simulated 401 failures produced no terminal notice; finishing at synthetic 30 seconds requested a 30-second charge without serviceFailure. convo.ts:2244-2268 catches out-of-band brain failures and recovers; this path does not enter agent.ts:335's SDK terminal/refund classifier. Learner input increments the billing predicate before the failed call.

Classify out-of-band errors, allow a bounded transient recovery and terminate definitive or sustained failures through the existing service-owned refund/settlement path. Typed fallback cannot fix an unavailable shared brain. **Acceptance:** voiced and typed sessions cover invalid model/key, repeated timeout, 429/5xx and transient recovery without charging unusable sessions contrary to the approved policy. **Owners:** voice/brain/revenue engineers.

A07 P1 Safety scans a prefix while the model receives the full input

Reproduced. safety.ts:130 inspects the first 4,000 characters. The protocol accepts longer text; a 4,138-character synthetic input put a policy-triggering suffix after benign padding, escaped the deterministic guard and reached a recorded mock frame request through the real conversation route. This proves policy-code evasion, not a successful real-model jailbreak.

Enforce strict shared schema/UTF-8 size limits before safety and send only the fully accepted/inspected input onward. Include topic, answer and provider transcript paths; reject rather than silently inspecting less than is dispatched. **Acceptance:** boundary, padding, Unicode/normalization and injection cases cannot send an uninspected suffix to providers. Run separate model/output red-team evaluations. **Owners:** voice/API/security/tutor-behavior engineers.

A08 P1 Shared TTS cache retains personalized speech indefinitely

Source verified. ttsCache.ts:149 accepts every nonempty reply up to 220 characters, including free-form tutor replies. Keys use voice/rate/text; cached WAVs have no owner, TTL, byte/file quota or account-erasure mapping. Hashing filenames does not remove spoken personal content. The published policy describes caching authored tutor lines, a narrower scope.

Restrict shared persistent caching to a versioned allowlist of authored nonpersonal lines; keep free-form replies ephemeral. Purge old cache data with a documented transition. If personalized caching is deliberately retained, implement owner binding, access controls, expiry, eviction and deletion semantics first. **Acceptance:** a synthetic private reply creates no global WAV, approved static lines still hit the cache, quota/expiry and old-cache cleanup are demonstrated. **Owners:** performance/privacy/backend engineers.

A09 P1 Direct Supabase writes bypass client and API resource bounds

Reproduced in the complete disposable schema. Authenticated own-row writes accepted a 614,477-byte learner payload despite the mobile 512 KiB cap, and 10,001 valid product events. 202608220001_luvani_learning.sql:10,21-28,47-49 provides owner access but no cumulative write quota. Client envelope checks and token-server rate limiting do not apply to direct Data API calls. Hosted gateway settings remain unknown.

Enforce byte/item/shape bounds and account storage/write/event quotas at every exposed database boundary. Either revoke direct event INSERT for an admission RPC/API or provide equivalent atomic DB controls. Keep legitimate offline sync semantics. **Acceptance:** direct authenticated REST/RPC over-limit writes fail; parallel writes cannot race quotas; response/query cost and retained volume stay bounded. **Owners:** data-sync/data/security engineers.

A10 P1 Account export fails above fixed size and row ceilings

Reproduced. The export fails above 10,000 rows in any owned table or 32 MiB total; 10,001 admitted product events triggered failure in the latest SQL. Those limits are useful memory safeguards, but the only portable-export route has no larger-account alternative. Retention is currently unenforced.

Provide a bounded asynchronous/paginated export with a coherent snapshot/cursor and temporary encrypted delivery, expiry and authorization. Reconcile its sixteen selected owned tables and exclusions, including account-matchable trial HMAC markers (accountExport.ts:8-9,92-95) and account-linked private queue work, against the policy's "every row" claim (apps/mobile/src/legal/policy.ts:211). A reviewed data inventory must define truthful scope; pagination alone does not complete it. Apply event limits/retention as well; merely raising the in-memory limit moves the failure. **Acceptance:** a

large synthetic account exports the approved scope without truncation or blocking API capacity, with owner/reauth controls, documented exclusions and expired-file cleanup. **Owners:** data/privacy/API engineers.

A11 P1 Account deletion depends on a billing processor even when billing is dark

Reproduced. productionConfigurationProblems permits an entirely unconfigured RevenueCat path. deleteAccountWithProcessor always calls processor erasure first; an absent key returns false with zero database requests. A trial-only deployment can therefore authenticate deletion but always answer 503. Configuring RevenueCat might mask the contradiction, not correct it.

Track whether an account has processor data; permit deletion when no processor relationship exists, and use a durable erasure workflow when it does. Handle partial processor/database completion and stop active sessions/provider work during deletion. **Acceptance:** billing-dark/trial-only deletion succeeds; paid processor failures are truthful and retryable; stale bearer/active-room tests cannot recreate data or dispatch new paid work after deletion. **Owners:** privacy/auth/API/revenue engineers.

A12 P1 Published retention promises lack deployed enforcement

Source verified. PRIVACY_AND_SAFETY.md:1026-1028 acknowledges missing scheduled deletion for promised analytics/usage/antifraud retention. No enforcement job was found. The audio-cache issue adds another retention inventory gap.

Implement indexed bounded retention jobs with run/failure metrics and documented backups/vendor expiry. Use server-owned receipt timestamps for expiry: client-writable product_events.occurred_at can be set in the future and evade a naive TTL. Reconcile the published 13/24-month promises (apps/mobile/src/legal/policy.ts:201) with permanent grant/refund proofs (:202). Separate expiring content/detail from the minimal permanent idempotency/completion proof required for debits, refunds, saved text and audio admission: blindly deleting the entire usage ledger can reopen replay or break saved learning. **Acceptance:** time-controlled expiry tests including future client timestamps, deployed run evidence, replay after retention, backup restore/deletion reapplication and policy reconciliation. **Owners:** privacy/data/revenue/deploy engineers.

A13 P1 Intended migrations and actual deployment have no verified compatibility contract

Live metadata and documented gap. The configured project is inactive; catalog/history requests time out. supabase/README.md:7-20 documents divergent remote versions and missing old history. Later feature migrations are explicitly prepared/not applied live. Current code requires new session-grant/refund/billing/export primitives.

Establish distinct staging/production targets, reconcile schema and migration history safely, and create a required-object/version manifest checked before promotion. Deploy with documented old/new writer compatibility. The refund migration specifically requires draining old grant/debit writers before changing lock order. **Acceptance:** clean replay, live role/catalog checks, simultaneous transaction races and complete canary journeys for the exact artifact/schema pair. **Owners:** deployment/data/revenue engineers and architects.

A14 P1 The hosted database verification harness is stale

Source verified. verify/supabaseLive.ts:158-177 inventories seven old tables and three old routines; deletion covers five tables. Debit fixtures call new random IDs without issuing grants, while the latest debit routine requires grant proof. It can reject the correct new schema while omitting major new features from its coverage.

Update the disposable/staging harness to the current object inventory and grant lifecycle. Include role privileges, saved text, social/block behavior, billing/refund/epoch races, queue deletion fences and all cascades. **Acceptance:** it passes on the intended latest schema and fails for a removed policy/grant/RPC or older implementation; proofs identify commit/project/schema and use independent PostgreSQL connections for races. **Owners:** data/QA/API engineers.

A15 P1 Backup claims and fallback coverage are unreliable

Source and primary-doc verified. The runbook's backup summary implies paid PITR/free daily backup and its CSV fallback lists only seven tables. Current Supabase backup documentation describes daily backups for paid plans, PITR as an add-on, and offsite exports for free projects. Database backups also do not contain Storage object bodies. Actual backup/PITR settings and a complete restore were not verified.

Correct the runbook and select a backup/recovery policy based on acceptable data loss and downtime. Cover all application schemas, auth/roles, queue state, artifacts/objects and secrets/config restoration. Restore to isolation and reapply post-backup erasures before use. **Acceptance:** a dated recoverable backup, full timed restore, integrity/permission/billing checks and documented RPO/RTO achieved. **Owners:** deployment/data/privacy engineers.

A16 P1 Public hosting and native release configuration are not demonstrated

Source/live-input verified, hosted environment unverified. The checkout has no service/container/host manifest or EAS profile; canonical mobile API points at localhost. A LiveKit Cloud URL and credentials do not prove an agent deployment. The current Supabase target is inactive. Production API guards correctly reject incomplete configuration.

Create reproducible deployment definitions for API, agent and practice worker; stable HTTPS/WSS domains; isolated envs; secret references; supported runtime; least-privilege process/network permissions and versioned mobile build inputs. **Acceptance:** a clean hosted staging deployment and cellular physical-device journey with no development fallback, then controlled production promotion. **Owners:** deploy/release/QA engineers.

A17 P1 CI succeeds but does not enforce the release boundary

Live metadata. Main reports unprotected with status-check enforcement off. CI runs on pushes/PRs and has a successful exact-head run, but it can detect a bad direct push only after it reaches main. No artifact deployment gate is present.

Require reviewed PRs and required checks on the protected production branch, with an audited emergency path. Private-repository plan restrictions may require a plan change or a documented alternative controlled release boundary; do not make the repository public merely to enable protection. Add environment approval, immutable artifacts and provenance/schema association for releases. **Acceptance:** an unreviewed/failing candidate cannot promote; a tested emergency rollback is possible. **Owners:** release/deploy engineers and repository owner.

A18 P1 Observability cannot establish service quality or detect money/privacy failures

Source verified, vendor/platform telemetry unverified. Console usage/timing exists, but no app-owned central tracing/error/metrics integration or request correlation was found. server.ts:256 returns healthy with no DB, Redis, agent or speech provider; the isolated fixture demonstrated this with dependencies absent. No actionable settlement/backlog/spend/error-budget alerts were evidenced.

Separate liveness, dependency readiness and capability status. Add scrubbed structured logs, request/session/job correlation, stage latency histograms, business ledger metrics, queue age/dead-letter visibility and alert ownership. Gate grants on known serving/admission readiness without turning probes into costly dependency fan-out per request. **Acceptance:** staged DB/Redis/agent/model failures produce correct readiness, zero unusable admitted work, an alert and a recoverable learner path. **Owners:** observability/voice/API/deploy engineers.

A19 P1 Voice and learning quality still require real release acceptance

Historical evidence with explicit limits. Local lifecycle tests establish hard stops and several zero-use/debit controls, not audible quality. REVAMP_B_CONTINUATION.md:660 records a twelve-call GPT-6 Luna sample at p50 4,864 ms/p95 8,661 ms; the earlier GPT-5.6 Luna pair is 3,928/8,208 ms. These are model-only samples and too small for a service SLO. Live scoring is default-off; free conversation explicitly has zero scored utterances. Physical microphone/network/background tests, representative beginner ASR and assessment calibration remain held.

Agree product claims, run consenting labeled French/bilingual audio evaluations and physical iOS/Android lifecycle tests with exact voices/models. Measure end-of-utterance to first audible reply, interruptions and regional transport under load. **Acceptance:** agreed workload/corpus thresholds, truthful pronunciation availability, real-device billing/refund parity and no regressions in quiet mode. **Owners:** voice/tutor/learning-science/QA engineers.

A20 P1 Whole-product vendor spend and admission need enforceable bounds

Source verified exposure; scale/cost unmeasured. API daily counters are helpful but do not bound aggregate account creation, voice data-channel generation, repeated replay/abandon work or practice-worker spend. Installation headers are client-provided strings, not hardware attestation. Saved-phrase audio has stronger hard budget reservations, but those intentionally apply only to that route.

Add global/account/vendor cost and concurrency budgets, real session reservations, per-job spend accounting and kill switches. Configure Supabase signup/reset defenses and verification/CAPTCHA based on staged abuse tests; add device attestation only if measured abuse warrants it. Reconcile observed usage with current vendor invoices, including failed/refused/retried calls. **Acceptance:** synthetic Sybil, command burst, replay and backlog tests hit a bounded circuit without bypass through other entry points; dashboard and invoice deltas are explainable. **Owners:** security/revenue/voice/data/deploy engineers.

A21 P2 JWT verification needs an explicit token contract and revocation policy

Reproduced with a trusted synthetic signing key. auth.ts:63 accepts a signed token with wrong issuer/audience, a nonlearner role or no expiry. It requires a nonempty subject but passes no claim expectations to jwtVerify. This is not evidence that an outsider can forge the project's signature. Expired tokens that do contain expiry are already rejected.

Define allowed issuer/audience/algorithms, mandatory expiry and subject/role/session requirements; evaluate anonymous users explicitly. For sensitive/costly operations check session/account revocation when necessary rather than assuming locally valid claims prove a live account. Supabase's JWT guide explains signing, claims and JWKS behavior. **Acceptance:** malformed/misissued/revoked/deleted-account cases fail without losing legitimate refresh/reconnect behavior. **Owners:** auth/security engineers.

A22 P2 Strict command schemas and bounded queues are incomplete

Reproduced malformed decoding, source-verified flood exposure. The protocol accepts known tags without validating all fields; numeric say produces a contained TypeError, not a worker crash. The turn queue has no pending bound; replay/hear commands can enqueue speech outside it. API budgets do not protect this data channel.

Use discriminated schemas, maximum bytes, sequence IDs, bounded pending work and per-session command/audio buckets. Prioritize leave/cancel over queued generation. **Acceptance:** malformed/burst/retry traffic produces no unnecessary provider work, bounded queue/memory and working normal accessibility/typing/reconnect flows. **Owners:** API/voice/security engineers.

A23 P2 Raw error logging can disclose learner-derived content

Reproduced Error inspection; source-verified log call sites. BrainParseError.raw is enumerable and contains the full failed model response; conversation post-session handlers and SDK error paths log raw errors. A synthetic marker appeared in inspection output. No real learner log leak was sampled.

Centralize safe logging fields and scrub SDK/provider failures; disable sensitive debugging in production. Include log access/retention in the privacy inventory. **Acceptance:** private synthetic markers in raw responses/errors never appear in production-captured logs while status/stage/correlation remain diagnosable. **Owners:** observability/privacy/backend engineers.

A24 P2 A hung sync request blocks subsequent account reconciliation

Reproduced with an injected never-settling transport. App.tsx:123 uses plain fetch and sync/engine.ts:314-334,630-638 serializes cycles without HTTP/body deadline or stop cancellation. A later reconcile/resume remained pending behind one request. The fresh-device UI has its own six-second boundary, so this is not an endless-startup-spinner claim.

Add request/body deadlines and abort on account stop/network change while preserving dirty state and owner fencing. **Acceptance:** a blackholed request expires and healthy foreground reconciliation subsequently converges without restarting the app. **Owners:** data-sync/mobile engineers.

A25 P2 Query pagination and aggregate cost need realistic-volume proof

Source verified. Conversation Pack listing (services/token-server/src/conversationPacks.ts:70-72; route server.ts:672-681) fetches an owner's entire result set with no cursor; PostgREST configured row limits can silently truncate that library. Saved conversation-text listing already implements a ten-item timestamp/session keyset cursor and exact-count handling (conversationText.ts:20-44). Social snapshots expand client JSON repeatedly and return incoming requests without recipient pagination. Four social foreign-key lookup columns lack suitable leading indexes in the local catalog.

Implement stable cursors/response limits for Conversation Packs and incoming social requests; preserve the saved-text pagination already implemented. Measure selective query/cascade plans. Review indexes for social_blocks.target, social_reports.user_id, social_reports.target and social_reactions.user_id; add only with query evidence. **Acceptance:** large libraries are complete across pages, Friends/deletion/export meet agreed p95 budgets and no payload creates unbounded aggregate scans. **Owners:** data/API/social/performance engineers.

A26 P2 Friends is self-reported and moderation is not operational

Reproduced and disclosed. A fabricated activity entry generated 100 points/one conversation with no grant or usage row. The privacy document explicitly calls points capped client-synced practice, not certified ability; this is not a billing or cross-user RLS breach. Sender caps exist, but recipient/global/Sybil admission, expensive-RPC quotas, request expiry and staffed report triage are not complete.

Keep competitive/reward claims consistent with self-report, or add immutable server completion receipts if verified achievements are desired. Establish moderator ownership and defensive pagination/recipient/pair quotas; keep Friends disabled at launch if this cannot be supported. **Acceptance:** coordinated incoming spam stays bounded, blocks/deletions cannot restore relationships, reports reach an actionable private staff workflow, and any claimed verified credit cannot be forged. **Owners:** social/security/privacy engineers.

A27 P2 Durable practice recovery needs consistent deadlines and operational deployment

Source verified. Recovery is flag-dependent and requires a separate running worker. It aborts drills at 60 seconds while historical healthy OpenAI batches motivated 90 seconds in the agent. Leases, five attempts, stale-result fencing and tombstones are strengths. The recovery path does not account for result usage or regenerate every memory/curation artifact.

Deploy and monitor the queue as part of the service release, align measured provider-aware deadlines, record job spend and visible exhausted/pending outcomes, and move necessary enrichment to durable owned work. **Acceptance:** a valid 65-second batch, concurrent claims, stale lease, kill/restart, duplicate receipt and deletion races behave correctly under the chosen budgets. **Owners:** backend/data/voice/revenue/deploy engineers.

A28 P2 Configuration and failure semantics need consistent production guards

Source verified. Voice startup probes lack deadlines; some unavailable results are nonfatal and can still produce an all-vendors-answered summary. Worker production reporting URL/agent secret are not fully enforced by preflight. Development CLI brains warn instead of being categorically prohibited. API null JSON returns 500, not 400. The API has no explicit graceful listener drain, and proxy IP extraction assumes exactly one trusted appending hop.

Validate the paired worker/API environment, supported provider/model, reporting authentication and actual edge topology. Bound probes, distinguish authentication from model/speech health, prohibit inappropriate local modes in deployed workers, validate bodies and drain active requests. **Acceptance:** intentional configuration errors fail closed at boot or readiness; invalid JSON yields 400; proxy-spoof and shutdown tests match deployment behavior. **Owners:** deploy/API/voice/security engineers.

A29 P2 Dependency acceptance and history scanning need stronger freshness guarantees

Reproduced policy behavior/source verified. Current audit passes with accepted high roots; adm-zip now has multiple advisories beyond the single originally listed URL, while the gate accepts by module name. The reachable-history scanner collects all Git patch output in a 256 MiB buffer and failed on this repository; uncaught error printing can include raw diff content. A passing tracked scan does not substitute for history or native-release bundle inspection.

Match decisions to advisory IDs/version/range and expiry/review conditions, review new advisories even for accepted packages, and preserve the existing tested reachability distinctions. Stream/chunk history scanning with sanitized errors, then scan exact release artifacts. Prefer targeted compatible upgrades; do not run blind npm audit fix --force. Review the relevant Supabase Postgres minor-upgrade notice when restoring/upgrading the configured older database; extension-specific remediation must be checked against the actual catalog. **Acceptance:** a new advisory in an accepted package causes re-review, large history scans complete without emitting matches, and supported runtime/native dependencies build reproducibly. **Owners:** security/deploy engineers.

A30 P2 Smaller integrity and cost-accounting edges remain

Source verified. Profile scalar merges can lose concurrent edits (sync/registry.ts:41-47); billed refusal usage is not always included in conversation activity totals; delayed hard-stop callbacks can produce duration above the debit endpoint's 900-second limit instead of clamping to the issued allowance. The conversation output guard does not cover every scripted SDK-generated reply. Fish's single configured voice is not an equivalent six-persona failover.

Use causal per-field preference semantics, account for all billed call outcomes, clamp authoritative usage to its grant and apply reviewed output policy consistently. Document actual fallback capabilities. **Acceptance:** reordered multi-device edits, refusal accounting, overloaded clocks and scripted harmful-output fixtures preserve data, billing and safety semantics. **Owners:** data-sync/revenue/voice/tutor engineers.

Proposed production architecture

Retain the current monorepo, shared core/brain, managed Postgres/Auth, Redis budgets and LiveKit. The recommendation is to make those boundaries dependable, not multiply services.

API and admission: stateless HTTPS API instances behind a verified edge/proxy, with strict route schemas, one exception boundary, request deadlines, overload backpressure and dependency readiness. Postgres owns an explicit session lifecycle (issued, admitted, ending, settled, revoked) with account/room/epoch/job fencing. Redis accelerates limits and leases; account money and spend reservations remain authoritative and reconcilable in the database. Define consistency behavior for Redis outage or loss.

Voice: persistent registered workers with bounded job admission and regional placement matched to users/providers. Each job validates shared state before provider construction, owns one live session and registers graceful finalization. Commands are bounded and cancellable. Fault classifiers cover both SDK and separate brain paths; actual health controls mode availability. No automatic switch of provider/model/voice without equivalent privacy, pedagogy, billing and compatibility proof.

Durable work: transactional admission/finish receipts and an outbox/reconciler for settlement, refunds, baseline receipts and required enrichment. The existing practice queue is the right seam for post-session drills; add only the missing guarantees and observability. At-least-once delivery with idempotent/fenced effects is the desired contract. Do not claim network delivery exactly once.

Data: local-first sync for account-owned learning data, with DB bounds and verified direct API access. Keep money, verified session completion, social verification if promised and service cost limits server-owned. Version each namespace/protocol and test supported older clients. Use stable keyset pagination for growing libraries/events. Avoid replicas/sharding until measured query/capacity limits justify them.

Audio/content: shared CDN/cache only for versioned authored content that contains no personal information. Personalized model speech is transient unless deliberate owner-scoped retention is implemented. Do not put user-scoped content into public cache keys or CDN defaults. Preserve the deliberately disabled saved-phrase generation switch until actual rates, caps and hosted concurrency/device acceptance are verified.

Operations: reproducible nonroot service artifacts with a pinned runtime, managed secrets, environment separation and immutable release IDs. Required CI checks validate the candidate, integration tests validate schema/role/contracts, canary checks validate the running service and database, and explicit abort/drain/recovery conditions control promotion. Rollback must account for database compatibility and data semantics, not just deploy an old binary.

Efficiency and quality measurements to implement

These are **proposed initial acceptance targets**, not measured results or universal standards. Adjust them after workload/cost trials, then commit the chosen launch targets and owners.

Metric	Proposed initial target or required decision	Measure
Valid API request service success	At least 99.9% over the agreed window, separating intentional refusals	Status/error class, route, region, release and dependency
Tutor availability after accepted grant	At least 99.5% usable starts	Admission-to-ready and actual first audible/typed reply
Session start	p95 at most 5 seconds to usable tutor under agreed network/load	Auth, DB, lease, dispatch, warm/cold job and transport stages
Voice turn reply	Initial goal p95 at most 3 seconds from learner end-of-turn to first audible reply	VAD endpoint, queue, STT, brain, safety, TTS and media delivery
Settlement and baseline	Zero permanently lost settled/required baseline effects; temporary pending bounded and alerted	Receipts, age, retry/fencing and reconciler outcomes
Queue health	Choose maximum oldest-job age and exhaustion action; alert before learner-visible loss	Queue depth, age, attempts, pending, leased, failed, job spend
Spend	Explicit account/global/vendor daily ceilings and shutdown conditions before launch	Successful, failed, refused, retried, cached and uncached work
Recovery	Decide maximum acceptable lost data and outage; prove the selected RPO/RTO in a restore	Backup lag, completeness, elapsed restore and reconciliation
Capacity	Measure 10, 50 and 100 concurrent-session staging tiers, then set launch cap with headroom	RSS/CPU, active jobs, event-loop delay, vendor throttles, queue age and DB latency

The p95 voice target is ambitious relative to the recorded model-only sample. It requires stage measurement and possibly shorter turns, model/routing changes or a redesigned safe generation path. Do not declare the target met by lowering a timeout. Do not use a twelve-call sample to estimate dependable p99.

Budget model: session cost equals STT + successful/failed/retried LLM + TTS + scoring + media + post-session recovery + infrastructure + support/refund overhead. Track distributions and worst permitted behavior, not only average normal sessions. Cache-hit ratios and actual invoices must validate savings; unrecognized model prices must remain explicitly estimated.

Implementation order and release exit conditions

Stage 1 Contain public attack and privacy exposure. Resolve A01-A04, bound commands and direct DB writes, correct the cache/log retention mismatch and test the billing-dark deletion path. Required exit: hostile synthetic requests preserve service availability, replay/abandon cannot expand admitted spend and no unapproved personal cache/log data persists.

Stage 2 Make money and earned learning durable. Implement A05-A06 lifecycle/outbox/reconciliation, safe export, retention-compatible idempotency proofs and consistent recovery deadlines. Required exit: kill/restart/uncertain-ACK and deletion races preserve exactly-once effects, usable practice and truthful refund status.

Stage 3 Establish the actual release environment. Reconcile hosted schema/history, update the verifier, deploy isolated staging with supported runtime/artifacts/secrets, configure auth anti-abuse and branch/release enforcement, and complete backup/restore. Required exit: catalog/role/race/canary checks and restore evidence identify the exact code/schema/environment.

Stage 4 Prove voice, performance and operations. Add readiness/telemetry/spend controls and perform real beginner/French quality, physical-device lifecycle, capacity and provider-outage acceptance. Required exit: agreed SLO and cost thresholds hold, alerts have an owner, and optional unverified features remain off.

Stage 5 Promote with controlled exposure. Release to a bounded cohort, review real invoice/settlement/backlog/privacy metrics, then expand admission under the measured cap. Use recorded stop/rollback criteria and compatible forward recovery. An active on-call/support owner and a tested incident path are part of readiness.

No calendar estimate is justified without assigned engineers, target load, hosting choices and remediation measurements. These stages specify dependencies and outcomes rather than invented delivery dates.

Backend architecture agents added for future changes and releases

Four read-only roles were added to the existing repository agent system:

Role	Specialty	Trigger
backend-architect	Cross-service ownership, protocols and state transitions	Substantial change spanning API, worker, client or data
database-security-architect	Auth, RLS/grants, money storage, migration and privacy	Data/auth/billing-schema/export/deletion change
voice-reliability-architect	Admission, SDK jobs, speech/typed input, settlement and recovery	Tutor/model/media/metering or practice-lifecycle change
deployment-architect	Exact commit/artifact/schema, hosting, SLO and restore	Backend promotion or hosting/release-control change

They are pinned in the existing gate and registered as read-only guards. /backend-review selects relevant roles; /forge consumes specialist advice for its design; /ship requests deployment and affected backend reviews. Existing territory engineers retain implementation ownership. The team and deployment instructions now reflect the current CI and Cloud/local distinction instead of stale blanket claims.

These are Claude Code repository agent definitions, with that repository's existing Fable model convention. They are not registered native Codex background agents, installed infrastructure, a CD service or a schedule. Other hosts must report unavailable pinned dispatch honestly. Adding the definitions does not prove their model is available or that a production review occurred. They explicitly prohibit review-side external mutations and do not grant ongoing deployment authority.

Excluding direct Edit/Write tools and instructing read-only Bash provides a review policy, not an operating-system write prohibition. Use an enforcing read-only sandbox when independence requires a hard boundary.

For each relevant change, hand reviewers the original requirement, raw diff, source, real environment and check results. Return evidence, blocker, owner, migration/rollout order and executable acceptance. Use an independent reviewer for implementation; a reviewer must not quietly repair the code it is judging. Record unknown launch-critical facts as HOLD, even with a green local gate.

Industry benchmark and limits

The accompanying benchmark covers a representative cohort of 100 major apps/services using published primary sources. It is not a ranked list of the world's 100 largest apps: no ranking, geography, store, time window or metric was specified. Company engineering posts and cloud customer case studies establish particular disclosed practices at their publication date; they do not reveal every service's complete present private stack. App-specific, shared-company, API/security and narrower evidence are labeled in the register. Requirements above come from Luvani's reproduced/source-verified risks and applicable published practices, not from popularity or copying large-company technology choices.

Completed targeted disclosure verification covered **100/100** distinct primary URLs: 94 Firecrawl page bodies and six primary web-reader fallbacks/corrections. Evidence comprises 61 app technical, 19 app security, six app API, three platform technical, seven company technical, two subsystem technical and two operations disclosures. These are targeted checks of the stated disclosure, not exhaustive whole-stack assessments.

The 25-practice synthesis provides cited requirements and maps them to A01-A30. The 100-entry CSV, structured register, validation counts and method preserve dates, scopes and assurance limits. Queue durability, regional media, reusable-content caching, release controls, privacy and recovery inform the priorities; current costs, topology and private provider settings require separate deployment evidence.

Reproducible evidence package

The evidence directory retains synthetic API, voice, JWT/deletion, complete-schema and hung-sync reproduction scripts, sanitized result files and independent draft reviews. Those reviews identified the publication precision corrections already incorporated here. It lists the seven existing SQL proof scripts and explains which fixtures simulate authentication/roles rather than exercise a public deployment. Full-gate validation also passed after the agent/routing changes: 4,232 tests, typechecks and model/command guards remain green.

Run these reproductions only against disposable local fixtures as documented. The oversized-audio and malformed-Host scripts intentionally stop their isolated child server; they are not probes for production. Raw secret-history errors, canonical environment contents, learner records and complete copied third-party pages are excluded.

Remaining verification and access limits

Production endpoints, dashboards, active hosted agent capacity, release build environment, actual auth policies/redirects/SMTP/CAPTCHA, signing-key configuration, deployed role grants, backup settings, provider retention and store billing were not attested. The matched inactive database was not restored. Hosted scripts that create/delete accounts or write learner/billing rows were not run. Paid model/audio quality calls, real invoice reconciliation and physical device/load tests were not run. Disposable PGlite does not substitute for independent simultaneous PostgreSQL connections, Supabase Auth or gateway behavior.

The architectural-agent changes were validated separately. Application remediation remains the report's implementation backlog. A green source gate is compatible with a public-release HOLD; readiness requires the additional acceptance evidence named above.

Industry practices and source comparison

Research date: 3 October 2026. This is a benchmark of **100 distinct app/service public disclosures**, not a ranked list of the world's 100 largest apps, an independent security certification, or an inspection of 100 complete private production architectures. The source register identifies technical articles, narrowly scoped security/API disclosures, operations disclosures, shared company evidence, and platform evidence separately. Historical articles demonstrate a pattern at publication time; they do not establish the app's complete current stack. Recommendations below are engineering inferences for Luvani and must be reconciled against the code and deployed configuration.

Practices that should change the launch standard

#	Public evidence	Implication for Luvani	Concrete completion criterion
1	OWASP identifies broken object authorization, broken property authorization, and unrestricted resource consumption as separate API risks. OWASP API Security Top 10	Treat database RLS, API ownership checks, mutable-field restrictions, and expensive-operation quotas as separate controls. RLS alone is insufficient.	Anonymous/user A/user B/admin tests cover every exposed table, storage object, RPC, and privileged endpoint; no caller-supplied owner or premium field can confer access.
2	Box documents API permissions following the same content permission model as its web application, with additional token scopes. Box API security	A frontend screen hiding another user's data is not authorization. Apply the same ownership rules to direct endpoints and service-role paths.	Direct HTTP calls using another user's IDs fail; service-role handlers derive authenticated identity rather than trust the request body.
3	LiveKit tokens encode room, identity, and grants. Expiration controls initial connection, and connected clients receive refreshed tokens. Removal/revocation behavior differs between Cloud and self-hosted deployments. LiveKit tokens and grants	A short token TTL is not a paid tutoring-session cutoff. Enforce an authoritative session state and stop the agent/media room when allowance ends. Avoid personal names/email in logged room identities.	Disconnect, reconnect, refreshed-token replay, duplicate start, app backgrounding, and expired entitlement all obey the server's duration and concurrency limits. Cloud revocation cutoff is tested explicitly; self-hosted rejoin prevention follows its documented limits.
4	Stripe explains using idempotency keys to make retries safe; Wise documents stable replay identifiers and bounded retry behavior. Stripe idempotency, Wise errors and retries	Credits, session admission, refunds, and billing reconciliation require unique operation IDs and atomic persistence. A retry after a timeout must not repeat a deduction or grant.	Parallel and repeated requests, process interruption, and duplicate delivery produce one durable entitlement/usage transition with a recoverable reconciliation record.
5	Stripe documents signature verification, duplicate events, and delivery-order limitations for webhooks. PayPal also documents signature validation. Stripe webhook delivery, PayPal integration security	Payment-provider events must be authenticated and deduplicated before mutating entitlement. Arrival order is not a reliable subscription state machine.	Stored event IDs are unique; invalid signatures are rejected; duplicate, delayed, and out-of-order events cannot restore a cancelled or superseded entitlement incorrectly.
6	Khan Academy described concurrent per-user writes and the need to protect consistent state. Khan Academy user write lock	Learning progress, streaks, usage, and lesson completion need atomic database transactions and uniqueness constraints. Do not copy custom distributed locks when a transaction suffices.	Two devices finishing the same lesson concurrently cannot double-award XP, lose newer progress, or make balance negative.
7	Slack describes durable asynchronous jobs with retries and operational tradeoffs; GitLab's architecture separates web serving from background processing. Slack job queue, GitLab architecture	Move post-session summaries, notifications, media generation, and reconciliation into durable jobs instead of untracked work after an HTTP response.	Jobs persist across worker death; retries are bounded; failed jobs can be inspected and replayed; queue age and backlog have alerts and an owner.
8	Amazon's engineering guidance explains timeout resource exhaustion, retry amplification, backoff, and jitter. Amazon timeouts, retries, and backoff	Put deadlines and concurrency budgets around STT, LLM, TTS, payment, and database calls. Retrying every layer can worsen an outage.	Provider latency/outage tests show bounded elapsed time, bounded retry count, controlled concurrency, and no growing orphan tutor workload.

#	Public evidence	Implication for Luvani	Concrete completion criterion
9	Netflix described protecting its API from cascading dependency failures. Life360 describes failure containment using cells. Netflix API resilience, Life360 resilience	An AI or analytics outage should not make authentication, saved lessons, or existing progress unusable. Separate critical from optional work and bound shared resources.	Disable each dependency in staging; core journeys remain usable where feasible, failures are clear to users, and resources recover without a deploy.
10	Discord moved voice capacity nearer users to reduce network distance. Teams documents ICE connectivity and media-relay call flows. Discord voice at the edge, Teams call flows	Voice needs regional media routing and tests through actual mobile networks, NATs, packet loss, and network switching. API uptime does not establish tutor audio quality.	Regional connection success, time to first audio, packet loss/jitter, reconnection, and agent availability are measured on supported devices and difficult network conditions.
11	LiveKit documents endpointing, interruption handling, preemptive generation, noise processing, and observability when tuning turn taking. LiveKit turn-taking tuning	Measure the entire voice pipeline, not only LLM latency. Faster speculative generation can waste compute and reply to incomplete transcripts.	Track STT delay, end-of-turn delay, LLM first token, TTS first audio, false interruption and cancellation rate; staged tests cover accented speech, pauses, cross-talk, noise, and long answers.
12	Duolingo explains CEFR-aware structured conversations, learning-designer instructions, dynamic evaluation, and controlled closers for Lily. Duolingo AI video-call design	A working audio connection is not proof of a working tutor. Version curriculum, prompt, model, tool contract, and assessment rubrics together.	A reviewed evaluation set tests level fit, factual accuracy, target-language practice, correction quality, off-topic handling, unsafe content, prompt injection, and useful session outcomes.
13	Duolingo's historical lesson-audio service generates files asynchronously, stores them in S3, and delivers them through CloudFront. Spotify describes standardizing CDN delivery. Duolingo reusable audio, Spotify CDN design	Cache reusable, non-personalized lesson audio rather than pay for synthesis on every playback. Dynamic private tutor speech has different privacy and cache requirements.	Content-addressed assets include language, voice, prompt/content version, and format; CDN permissions, invalidation, hit ratio, and cost are documented.
14	Tinder describes cache-aside reads with a durable source of truth. Reddit's Devvit cache explicitly warns against personalized or sensitive shared responses. Tinder cache-aside, Reddit shared-cache boundaries	Cache only data with defined isolation and staleness rules. Cached premium state or user progress can cause access/correctness failures.	Keys include required tenant/user/version dimensions; TTL and invalidation are explicit; cache-miss/failure load is tested; private data never reaches a shared cache key.
15	Figma documents PostgreSQL connection safeguards in PgKeeper; Notion describes planned capacity migration. Figma PostgreSQL protection, Notion database migration	Bound connection pools, inspect slow queries, index authorization filters, paginate history, and validate migration compatibility before adding databases or sharding.	Expected peak load passes with measured query plans, connection headroom, p95/p99 latency, stable write behavior, and rollback-compatible migration sequencing.
16	Peloton describes sudden synchronized session-start traffic and preparation for large live events. Pokémon GO disclosed launch traffic far beyond forecast. Peloton event engineering, Pokémon GO launch	Marketing spikes hit token issuance, agent starts, provider quotas, database writes, and media concurrently. Average requests per second is insufficient.	Model sustained traffic and sudden joins; verify voice/AI quota reservations, worker warm capacity, admission control, database headroom, and user-facing overload behavior.
17	Canva's case study describes centralized security-log collection and investigation. Cambly exposes separate operational components, including login and lesson transcripts. Canva security operations, Cambly status	Observe critical user journeys and security events separately. Logs alone do not imply alerting, ownership, or usable incident response.	Dashboard and alerts cover login, entitlement mutation, tutor start/finish, summary completion, provider errors, queue age, and suspicious access; request/session IDs correlate traces without exposing raw audio or tokens.
18	Uber describes integration test gates for code and configuration. Shopify describes merge queues, canary deploys, and canary analysis. Uber test gates, Shopify release culture	Git history and green CI do not by themselves stop unsafe production changes. Protect the release path, test actual deployed artifacts, and include configuration/prompt changes in it.	Required checks apply to the commit being released; protected branches and review rules are enforced; staged canary and rollback are tested for API, agents, migrations, and tutor configurations.
19	Evernote discloses least-privilege production network boundaries and Terraform infrastructure configuration. Evernote security	Infrastructure, scheduled jobs, secret names, quotas, and alerts should be reproducible and reviewed, with controlled production administration.	A clean environment can be recreated from versioned manifests plus approved secrets; privileged access is attributable, limited, and logged.

#	Public evidence	Implication for Luvani	Concrete completion criterion
20	Monzo describes an intentionally smaller independent fallback for critical payments. GitHub describes database failover routing and limitations. Monzo fallback, GitHub database HA	Backups are only evidence of backup creation. Prove restoration of database, object storage references, auth configuration, secrets, and worker settings; define graceful degraded learning behavior.	A dated restore exercise measures achievable RPO/RTO and validates login, lessons, entitlement, progress, and transcript linkage in the restored environment.
21	Signal designs for metadata minimization. Memrise discloses pronunciation/communication voice recordings and progress storage. Airtable discloses subprocessor review. Signal sealed sender, Memrise privacy, Airtable subprocessors	Define why each audio/transcript field exists, retention duration, deletion propagation, region, and provider access. Encryption does not remove unnecessary retained personal data.	A data inventory covers app, database, storage, logs, backups, AI, speech, and analytics providers; deletion/export workflows and retention jobs are tested, including account deletion during an active session.
22	Google Meet distinguishes end-to-end, cloud, and client-side encryption. Messenger separately describes encrypted transport/history protocols. Meet encryption, Messenger end-to-end security	Document which app, media relay, agent, and AI service can decrypt content. Do not label a server-processed AI conversation end-to-end private merely because WebRTC transport is encrypted.	The data-flow diagram identifies decryption and storage boundaries; product privacy statements agree with the actual voice and transcript processing path.
23	SoundCloud describes backend APIs shaped for specific client needs; Asana describes realtime backend semantics supporting its work graph. SoundCloud service architecture, Asana tech stack	Evaluate the backend against app journeys: onboarding, lessons, tutor start, feedback, subscription, offline/reconnect, deletion, and cross-device progress. Technology inventory alone misses broken feature contracts.	Each feature has an API contract, authoritative state owner, failure behavior, and end-to-end acceptance check; hot screens avoid unnecessary sequential calls.
24	Babbel describes shifting management work to ECS Fargate/Lambda. Threads reused established Instagram infrastructure instead of rebuilding everything. Babbel managed compute, Threads reuse	A top-tier result does not require microservices, Kubernetes, Kafka, or multiple clouds. A small managed modular backend can be production quality if authorization, correctness, observability, recovery, and limits are proven.	Choose additional infrastructure only for a measured bottleneck or reliability requirement, with an owner, cost model, and migration plan.
25	OWASP MASVS covers storage, cryptography, authentication, network, platform, code, resilience, and privacy; its testing guide ties controls to verification. OWASP MASVS, OWASP mobile testing	The app security review must also cover mobile token storage, logging, network trust, dependency hygiene, deep links, and sensitive offline caches alongside backend RLS.	A tailored MASVS checklist has evidence and ownership, with high-risk gaps independently tested before public launch.

Architectural guidance

For Luvani, a reasonable target is a managed Postgres/Auth/Storage control plane, a small typed API layer, a durable job mechanism, managed WebRTC media, bounded tutor workers, and centralized metrics/error tracking. Separate the **control plane** (identity, entitlement, room admission, session state, progress, usage), **media/agent runtime** (audio, turn handling, model and speech calls), and **post-session processing** (summary, assessment, notifications, reconciliation). This is a recommendation, not a statement that the current code already implements these boundaries.

The most valuable investment before launch is proving negative and failure cases: cross-user access denial, immutable premium/credit fields, replayed billing events, concurrent sessions, orphan workers, provider timeouts, reconnects, failed summaries, and restoring a usable account. Copying a famous app's technology without its operating discipline would not establish production readiness.

Evidence limits and open questions

- Sources disclose a component, integration, or stated policy. A security-policy source has weaker assurance than a detailed engineering post and neither substitutes for an independent test.
- Shared company examples (such as King/Supercell or Expedia Group) are marked as shared. Their source does not establish a separately verified architecture for every branded app.

- Platform references (Open edX, GitLab architecture, Reddit Devvit) describe the platform; deployment-specific settings remain unknown.
- Historical examples include Duolingo lesson audio (2017), Khan Academy concurrency (2016), and Quizlet migration (2016). Their patterns are relevant; their current private stacks cannot be inferred.
- Public customer case studies can be selective and vendor-sponsored. Reported performance/cost outcomes are treated as the publisher's disclosure, not independent benchmark measurements.
- Neither this industry register nor a repository review proves deployed RLS, dashboard flags, branch rules, provider quotas, billing configuration, backup retention, or real tutor behavior unless those are inspected separately.

The completed register contains 100 distinct app/service entries and 100 distinct primary source URLs, with targeted disclosure verification for every entry: 94 used Firecrawl page content and 6 used primary-source content through the web reader after access failure or source correction. Scope counts: 61 app technical, 19 app security, 6 app API, 3 platform technical, 7 company technical, 2 subsystem technical, 1 app operations, and 1 company operations. No complete private production architecture was inspected. Source scope and review depth should accompany any claim that 100 apps were studied.

Mapping the benchmark to this audit

These are recommendations derived for Luvani, not claims that copying a particular company's implementation resolves its defects. Finding identifiers refer to BACKEND_PRODUCTION_AUDIT.md.

Practice area	Luvani findings	First concrete implementation
Hostile-request containment and authorization	A01, A02, A09, A21, A22, A28	One async error boundary, strict schemas, token contract and database write quotas
Authoritative voice admission and cost	A03, A04, A20, A30	Persisted fenced session lifecycle and bounded account/global provider reservations
Idempotency and durable jobs	A05, A06, A27	Awaited finalization, transactional receipts/outbox and observable reconciliation
Voice and tutoring outcomes	A07, A19, A22, A28, A30	Complete input checks, French/device evaluations and stage latency telemetry
Data and privacy lifecycle	A08, A10, A11, A12, A23, A26	Authored-only shared audio, complete export/erasure inventory and server-timestamp retention
Database correctness and efficiency	A09, A13, A14, A24, A25, A30	Current live-schema harness, compatible migration sequencing, deadlines and selective pagination/index proof
Release, recovery and observability	A13, A15, A16, A17, A18, A29	Immutable staging/release artifacts, required checks, timed restore and owned alerts

100-app/service source register

Exactly 100 distinct cohort entries. Each citation establishes only the stated disclosure and scope. Historical or policy evidence is qualified in the editable CSV; publication date can be unknown. Source labels are clickable; complete titles and validation details are preserved in the CSV.

Cohort entries 1-10

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
1 / Duolingo	app-technical	Pre-generates reusable lesson audio and persists audio metadata. Cache versioned reusable tutor audio; distinguish dynamic conversations from fixed lesson speech.	Powering Language Learning on Duolingo with Amazon Polly | Artificial Intelligence Published: 2017-05-12 Reviewed: 2026-10-03
2 / Babbel	app-technical	Describes production migration to ECS Fargate and Lambda. Managed compute can reduce operational work without abandoning controlled deployments.	Choosing serverless: Babbel's migration story | Containers Published: 2021-06-18 Reviewed: 2026-10-03
3 / Busuu	app-security	Documents SAML 2.0 service-provider SSO for business learners. Support institutional authentication through standards if Luvani adds school/business accounts.	Administrator - EN - Management platform with teams and live lessons Published: Not determined Reviewed: 2026-10-03
4 / Memrise	app-security	Privacy policy describes payment information supplied to external payment providers. Privacy-disclosure evidence only; minimize sensitive payment data stored by the app.	Privacy Page Published: Not determined Reviewed: 2026-10-03
5 / Quizlet	app-technical	Describes infrastructure migration and serving learning traffic on Google Compute Engine. Benchmark learning workloads before selecting scaling mechanisms.	Free online learning tool serves 200,000 requests per minute on Google Cloud Platform | Google Cloud Blog Published: 2016-03-10 Reviewed: 2026-10-03
6 / Khan Academy	app-technical	Describes concurrency control for user learning data. Protect progress updates from lost writes and concurrent completion requests.	The User Write Lock: an Alternative to Transactions for Google App Engine - Khan Academy Blog Published: 2016-06-20 Reviewed: 2026-10-03
7 / Coursera	app-security	Discloses security program based on ISO 27001 and third-party audit. Use external assurance and documented security program as maturity evidence; complete architecture is not disclosed.	Security at Coursera | Coursera Published: Not determined Reviewed: 2026-10-03
8 / Udemy	app-security	Discloses outsourced card processing without storing full card information. Keep payment card data outside Luvani and reconcile entitlements from verified provider events.	Credit card security - Udemy Published: Not determined Reviewed: 2026-10-03
9 / edX / Open edX	platform-technical	Public platform architecture separates LMS and course authoring with stable APIs. Separate learner access from content administration and define explicit API contracts.	Open edX Platform Architecture - Latest documentation Published: Not determined Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
10 / Brilliant	app-security	Documents restricting classroom progress visibility to the teacher and individual student. Model teacher/student permissions explicitly; policy evidence is not an RLS implementation audit.	Can teachers assign Brilliant lessons and monitor student progress? | Brilliant Published: Not determined Reviewed: 2026-10-03

Cohort entries 11-20

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
11 / Speak	app-technical	OpenAI identifies Speak role-play as a Realtime API use case. Realtime voice needs interruption handling and transport reliability alongside model quality.	Introducing the Realtime API | OpenAI Published: 2024-10-01 Reviewed: 2026-10-03
12 / Cambly	app-operations	Publishes separate operational components for login video chat text chat and lesson transcripts. Monitor each tutor dependency and publish incident status.	Cambly Status Published: Not determined Reviewed: 2026-10-03
13 / ELSA Speak	app-API	API accepts speech and returns pronunciation assessment for scripted and spontaneous speech. Implement speech assessment as a defined scored workflow rather than only conversational chat.	ELSA | API Published: Not determined Reviewed: 2026-10-03
14 / Calm	app-technical	Describes managed personalized content recommendations. Use event quality and evaluation to improve lesson recommendations.	Personalizing wellness recommendations at Calm with Amazon Personalize | Artificial Intelligence Published: 2021-09-23 Reviewed: 2026-10-03
15 / Headspace	app-security	Discloses managed identity for growing consumer and enterprise-customer access. Keep identity implementation managed; enforce authorization separately for every data path.	Auth0 | How Headspace Scaled Identity and Minimized Security Risk Published: Not determined Reviewed: 2026-10-03
16 / Strava	app-technical	Describes AI activity insights with model selection and content guardrails. Tutor responses require output quality and safety evaluation at scale.	Strava safely transforms the athlete experience using Amazon Bedrock | Case Study | AWS Published: Not determined Reviewed: 2026-10-03
17 / Peloton	app-technical	Describes architecture and preparation for large live fitness events. Load-test simultaneous live sessions and measure interactive experience under peak load.	How Peloton Engineers the World's Largest Live Fitness Events on AWS | AWS for Industries Published: 2026-07-20 Reviewed: 2026-10-03
18 / Fitbit	app-API	Documents authenticated data API and rate-limit response headers. Use explicit quotas and authorization on user-specific data APIs.	Get Activity Intraday by Date Range Published: Not determined Reviewed: 2026-10-03
19 / MyFitnessPal	company-technical	Under Armour case study says MyFitnessPal already ran on AWS; detailed migration concerns other connected-fitness apps. Shared portfolio evidence proves hosting context only; no app-specific security claims follow.	under-armour Published: Not determined Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
20 / Life360	app-technical	Describes multi-AZ autoscaling and migration to multiple clusters/cells. Limit incident blast radius; choose added topology only when operationally justified.	Life360's journey to a multi-cluster Amazon EKS architecture to improve resiliency | Containers Published: 2023-06-28 Reviewed: 2026-10-03

Cohort entries 21-30

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
21 / WhatsApp	app-technical	Describes end-to-end encrypted multi-device session design. Session identity and multi-device synchronization require explicit security design.	How WhatsApp enables multi-device capability - Engineering at Meta Published: 2021-07-14 Reviewed: 2026-10-03
22 / Messenger	app-security	Describes end-to-end encrypted messaging and secure storage. Define which private content servers and vendors may access; protect retained conversation data.	Building end-to-end security for Messenger - Engineering at Meta Published: 2023-12-07 Reviewed: 2026-10-03
23 / Instagram	company-technical	Threads article describes reusing Instagram backend models business logic security and infrastructure. Shared-company/backend evidence; not an independent inspection of Instagram's complete current stack.	Threads: The inside story of Meta's newest social app - Engineering at Meta Published: 2023-09-07 Reviewed: 2026-10-03
24 / Facebook	app-technical	Describes graph storage with MySQL and distributed cache. Cache hot reads while maintaining an authoritative durable database.	TAO: The power of the graph - Engineering at Meta Published: 2013-06-25 Reviewed: 2026-10-03
25 / Threads	app-technical	Describes social graph storage and serverless asynchronous compute used for Threads. Separate latency-sensitive user requests from deferrable processing.	How Meta built the infrastructure for Threads - Engineering at Meta Published: 2023-12-19 Reviewed: 2026-10-03
26 / Discord	app-technical	Describes regional edge voice infrastructure and real-time constraints. Measure end-to-end voice latency and route sessions near users.	How We Moved Discord Voice to the Edge Published: 2026-06-09 Reviewed: 2026-10-03
27 / Snapchat	app-technical	Describes low-latency messaging redesign and tiered media storage. Choose storage lifecycle by access frequency and cache media independently of live sessions.	AWS Innovator: Snap | Case Studies, Videos and Customer Stories Published: Not determined Reviewed: 2026-10-03
28 / Reddit	platform-technical	Devvit documents server-side shared cache helper. Platform API evidence only; use bounded cache TTLs and avoid repeated expensive computations.	Cache Helper | Reddit for Developers Published: Not determined Reviewed: 2026-10-03
29 / TikTok	app-security	Describes security gateways restricting employee access and data transmission. Restrict administrative data access and audit cross-region transfers.	Progress update on Project Clover - Newsroom | TikTok Published: Not determined Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
30 / LinkedIn	app-technical	Describes event streaming with operational availability constraints. Make progress events durable and replayable when downstream processing fails.	How LinkedIn customizes Apache Kafka for 7 trillion messages per day Published: 2019-10-08 Reviewed: 2026-10-03

Cohort entries 31-40

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
31 / X / Twitter	app-technical	Describes user database scaling with SLOs and encrypted transport. Use measured read load latency and consistency requirements to drive database changes.	How we scaled Reads On the Twitter Users Database Published: 2023-02-23 Reviewed: 2026-10-03
32 / Telegram	app-security	Documents different encryption models for cloud chats and secret chats. Describe transport encryption and end-to-end encryption accurately for voice features.	Telegram FAQ Published: Not determined Reviewed: 2026-10-03
33 / Signal	app-security	Describes minimizing retained user metadata and hiding sender metadata. Minimize audio transcripts and personal-data retention before adding more encryption layers.	Signal >> Blog >> Technology preview: Sealed sender for Signal Published: 2018-10-29 Reviewed: 2026-10-03
34 / Pinterest	app-technical	Describes managed infrastructure supporting recommendation and moderation models. Evaluate relevance and responsible AI quality as production metrics.	How Pinterest pushes boundaries of AI-powered discovery using AWS Published: Not determined Reviewed: 2026-10-03
35 / Tinder	app-technical	Describes cache-aside reads backed by durable source-of-truth storage. Cache bounded read-heavy data with failure paths and invalidation policy.	Building resiliency at scale at Tinder with Amazon ElastiCache | AWS Database Blog Published: 2020-01-30 Reviewed: 2026-10-03
36 / Bumble	app-security	Discloses confidentiality protection and secured servers/firewalls. Privacy-policy evidence only; do not equate stated measures with independently verified effectiveness.	Bumble Privacy Policy Published: Not determined Reviewed: 2026-10-03
37 / Hinge	app-security	Describes authenticated user data export with limited download availability. Implement data export with ownership checks and expiring access.	How can I make a Privacy Request? - Hinge Published: Not determined Reviewed: 2026-10-03
38 / Grindr	app-technical	Describes completed backend infrastructure modernization and migration. Maintain recorded infrastructure change plans and validate their operational results.	Grindr Backend Tech: 2023 Achievements & 2024 Roadmap | Grindr Published: 2024-02-06 Reviewed: 2026-10-03
39 / Slack	app-technical	Describes durable asynchronous jobs and service quality tradeoffs. Use queued retryable jobs for transcripts summaries notifications and billing reconciliation.	Scaling Slack's Job Queue | Engineering at Slack Published: 2017-12-06 Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
40 / Zoom	app-security	Documents optional E2EE and meeting access controls. Explicitly authorize session participants and document encryption boundaries.	Getting started with Zoom security and privacy Published: Not determined Reviewed: 2026-10-03

Cohort entries 41-50

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
41 / Microsoft Teams	app-technical	Documents ICE media connectivity and transport relays. Test NAT firewall and mobile-network failures rather than only successful local voice calls.	Microsoft Teams call flows - Microsoft Teams | Microsoft Learn Published: Not determined Reviewed: 2026-10-03
42 / Google Meet	app-security	Documents different encryption models for calls and meetings. State precisely which participants and server services can decrypt audio.	Learn about call & meeting encryption in Google Meet - Google Meet Help Published: Not determined Reviewed: 2026-10-03
43 / Netflix	app-technical	Describes protecting API availability from dependency failures. Use timeouts resource isolation and useful fallback behavior.	Making the Netflix API More Resilient | by Netflix Technology Blog | Netflix TechBlog Published: 2017-04-18 Reviewed: 2026-10-03
44 / Spotify	app-technical	Describes coordinated CDN strategy for playback. Deliver reusable media through CDN and measure playback quality.	How Spotify Aligned CDN Services for a Lightning Fast Streaming Experience | Spotify Engineering Published: Not determined Reviewed: 2026-10-03
45 / Prime Video	app-technical	Describes scaling video download metadata with durable distributed storage. Treat offline media entitlement and download state as authoritative backend data.	prime_video_dynamodb Published: Not determined Reviewed: 2026-10-03
46 / Hulu	app-technical	Discloses live TV infrastructure handling traffic spikes. Capacity-plan launch spikes and interactive usage independently of average traffic.	Hulu Case Study Published: Not determined Reviewed: 2026-10-03
47 / Disney+	app-technical	Describes global tables for watchlists recommendations and video bookmarks. Synchronize user progress across devices while defining consistency and failover behavior.	Amazon DynamoDB use cases for media and entertainment customers | AWS Database Blog Published: 2024-06-26 Reviewed: 2026-10-03
48 / Peacock	app-technical	Describes deliberately simplified scalable highly available launch infrastructure. Prefer provable reliable core user journeys for launch.	Peacock Case Study Published: Not determined Reviewed: 2026-10-03
49 / Paramount+	app-technical	Describes active-active regional infrastructure and SRE adoption. Choose recovery strategy from user-facing availability requirements.	Paramount Global built its streaming platform on Google Cloud | Google Cloud Blog Published: 2024-05-09 Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
50 / HBO Max / Max	app-security	Documents account-security guidance including reviewing and removing unused devices. Support attributable session/device management and recovery; guidance does not disclose private backend architecture.	Recommendations and tips to help keep your HBO Max account secure Published: Not determined Reviewed: 2026-10-03

Cohort entries 51-60

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
51 / YouTube	app-technical	Documents original YouTube MySQL scaling problems that led to Vitess. Begin with sound relational design; add sharding only when measured scale demands it.	The Vitess Docs | History Published: Not determined Reviewed: 2026-10-03
52 / Twitch	app-technical	Describes distributed regional ingest and media proxy redesign. Treat media transport as a distinct failure domain and measure regional session quality.	Ingesting Live Video Streams at Global Scale Published: Not determined Reviewed: 2026-10-03
53 / SoundCloud	app-technical	Describes APIs shaped to specific client needs through backend-for-frontend services. Make backend APIs complement actual mobile screens and reduce unnecessary request fanout.	Service Architecture at SoundCloud - Part 1: Backends for Frontends | SoundCloud Backstage Blog Published: Not determined Reviewed: 2026-10-03
54 / Zen by Deezer	app-technical	Describes cloud-native wellness service with managed cloud resources. Managed services can support small-team development; does not establish the Deezer music app's entire architecture.	Zen by Deezer Case Study | Google Cloud Published: Not determined Reviewed: 2026-10-03
55 / Roblox	app-technical	Describes cell architecture and migration testing. Test and isolate backend workloads to limit cascading incidents.	How We're Making Roblox's Infrastructure More Efficient and Resilient Published: 2023-12-07 Reviewed: 2026-10-03
56 / Fortnite	subsystem-technical	Describes internal game analytics search performance tuning. Separate analytics workload tuning from latency-critical user/session traffic.	How a team at Epic Games tuned Amazon OpenSearch Service for Fortnite analytics | AWS Big Data Blog Published: 2026-08-10 Reviewed: 2026-10-03
57 / Clash of Clans	company-technical	Supercell discloses managed infrastructure serving its game portfolio including Clash of Clans. Shared game-company evidence; no claim about a separately verified app-specific architecture.	AWS Innovator: Supercell | Case Studies, Videos and Customer Stories Published: Not determined Reviewed: 2026-10-03
58 / Candy Crush Saga	company-technical	King describes global delivery of content for its games including Candy Crush. Shared company CDN evidence; use origin/CDN economics and regional delivery metrics.	King - Using Amazon CloudFront to Deliver Mobile Games to Over 200 Countries | AWS News Blog Published: 2015-08-06 Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
59 / Pokemon GO	app-technical	Describes launch traffic far exceeding forecast and scaling response. Model viral demand and establish provider escalation paths before public launch.	Bringing Pokémon GO to life on Google Cloud | Google Cloud Blog Published: 2016-09-29 Reviewed: 2026-10-03
60 / Pokemon TCG Pocket	app-technical	Japanese case study describes Spanner for global launch and compute efficiency work. Validate global data consistency and cost per session; architecture disclosure is app-specific.	Pokemon TCG Pocket primary disclosure Published: Not determined Reviewed: 2026-10-03

Cohort entries 61-70

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
61 / Figma	app-technical	Describes connection-pool control and safeguards around PostgreSQL. Bound database connections and protect database capacity from application surges.	PGKeeper: Building the Bouncer We Needed for Postgres | Figma Blog Published: 2026-05-04 Reviewed: 2026-10-03
62 / Canva	app-technical	Describes centralized security log collection threat detection and investigation. Keep security-relevant logs searchable with alerting ownership and retention limits.	canva-professional-services Published: Not determined Reviewed: 2026-10-03
63 / Notion	app-technical	Describes capacity increase through tested database re-sharding without downtime. Plan and validate schema/data changes so deployments do not break clients.	The Great Re-shard: adding Postgres capacity (again) with zero downtime Published: 2023-07-17 Reviewed: 2026-10-03
64 / Trello	app-technical	Describes event-driven service and separate ephemeral shared data. Use real-time state channels for responsive UX while keeping durable records authoritative.	The Trello tech stack - Inside Atlassian Published: 2012-01-19 Reviewed: 2026-10-03
65 / Asana	app-technical	Describes realtime backend supporting its work graph. Define event synchronization and permission semantics around client features.	Asana's Tech Stack: How we build a collaborative app for teams of all sizes - The Asana Blog • Asana Published: Not determined Reviewed: 2026-10-03
66 / monday.com	app-technical	Describes changing serving database architecture to meet measured complex read needs. Optimize workload shape before introducing a new datastore.	mondayDB 3 - Solving HTAP for a Trillion-Table System - monday AI engineering Published: 2026-05-19 Reviewed: 2026-10-03
67 / Airtable	app-security	Discloses review of vendor privacy security and confidentiality before engagement. Record AI/voice subprocessors data location and access obligations.	Airtable Subprocessors | Airtable Published: Not determined Reviewed: 2026-10-03
68 / Dropbox	app-security	Describes optional end-to-end encrypted team files and key management. Design encryption around actual data-sharing and AI-processing requirements.	Implementing end-to-end encryption for Dropbox teams - Dropbox Published: 2024-06-05 Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
69 / Box	app-API	API follows web application content permissions and explicit token scopes. Use identical ownership checks on direct API and frontend access paths.	Security - Box Dev Docs Published: Not determined Reviewed: 2026-10-03
70 / Evernote	app-security	Discloses production network boundaries least privilege and Terraform infrastructure definition. Keep infrastructure changes versioned and administrative production access restricted.	Security Overview | Evernote Published: Not determined Reviewed: 2026-10-03

Cohort entries 71-80

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
71 / ClickUp	app-security	Discloses encryption vendor reviews key governance and geographic replication. Manage operational security controls and verify their real deployment.	ClickUp™ | Security Policy Published: Not determined Reviewed: 2026-10-03
72 / GitHub	app-technical	Describes database failover routing detection and known failure limits. Test recovery paths and acknowledge achievable RPO/RTO rather than promising zero data loss.	MySQL High Availability at GitHub - The GitHub Blog Published: 2018-06-20 Reviewed: 2026-10-03
73 / GitLab	platform-technical	Documents web/API serving PostgreSQL Redis and Sidekiq job processing. Separate interactive request handling from asynchronous work with explicit monitoring.	GitLab architecture overview | GitLab Docs Published: Not determined Reviewed: 2026-10-03
74 / Shopify	app-technical	Describes merge queue canary deployment and automatic canary analysis. Use deployment gates gradual rollout and reliable rollback.	Software Release Culture at Shopify - Shopify Published: 2020-04-15 Reviewed: 2026-10-03
75 / Etsy	app-technical	Describes migration from mature sharded MySQL to Vitess. Treat database migration as engineered operational change with validation.	Etsy Engineering | Migrating Etsy's database sharding to Vitess Published: 2026-03-19 Reviewed: 2026-10-03
76 / eBay	app-technical	Describes aggregating HTTP API requests to reduce network latency/bandwidth. Reduce redundant client/backend round trips and avoid request fanout.	Announcing ql.io Published: 2011-11-30 Reviewed: 2026-10-03
77 / Amazon Shopping	app-technical	Describes backend services supporting Prime Day traffic and fault-injection preparation. Validate peak traffic resource quotas and failure recovery before marketing spikes.	AWS services scale to new heights for Prime Day 2025: key metrics and milestones | AWS News Blog Published: 2025-08-26 Reviewed: 2026-10-03
78 / Stripe	app-API	Documents safe retry semantics using stable idempotency keys. Use idempotency and transactional deduplication for credit usage billing and subscription events.	Designing robust and predictable APIs with idempotency Published: Not determined Reviewed: 2026-10-03
79 / Mercado Libre	app-technical	Discloses scalable commerce and analytics infrastructure for large traffic peaks. Instrument checkout/entitlement correctness independently of overall system uptime.	AWS Innovator: Mercado Libre | Case Studies, Videos and Customer Stories Published: Not determined Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
80 / Zalando	app-technical	Describes efficient reliable streaming data-sync architecture. Make learning analytics/derived progress replayable and monitor event lag.	Zalando Reduces Cloud Costs for Tracking Platform by 4x and Latency by 40x Using Amazon Managed Service for Apache Flink Published: Not determined Reviewed: 2026-10-03

Cohort entries 81-90

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
81 / Zalora	app-technical	Describes CDN WAF DDoS protection and DNS integration. Combine CDN caching with edge abuse protection; retain application authorization checks.	Zalora Boosts Site Performance by 46% by Migrating its Content Delivery Network to AWS Published: Not determined Reviewed: 2026-10-03
82 / Uber	app-technical	Describes integration-test gates for code and configuration changes. Gate security billing and tutor lifecycle regressions before deployment.	Shifting E2E Testing Left at Uber Published: Not determined Reviewed: 2026-10-03
83 / Lyft	app-technical	Describes autoscaling event streams and durable ride tracking. Scale stateless serving while preserving durable session/progress records.	lyft Published: Not determined Reviewed: 2026-10-03
84 / DoorDash	app-technical	Describes transparent backend cache reducing repeated remote calls. Measure cache value and do not cache identity entitlements or mutable private state incorrectly.	Building a transparent high-performance proxy cache for DoorDash services - DoorDash Published: 2026-06-22 Reviewed: 2026-10-03
85 / Instacart	company-operations	Describes structured on-call investigation with persistence and evidence constraints. Maintain searchable evidence and tested incident investigation workflows.	Blueberry: Force Multiplier For The On-Call Engineer Published: 2026-07-14 Reviewed: 2026-10-03
86 / Deliveroo	app-technical	Describes scalable infrastructure supporting fluctuating delivery demand. Capacity-plan variable demand with user-facing service quality metrics.	Deliveroo on AWS: Case Studies, Videos, Innovator Stories Published: Not determined Reviewed: 2026-10-03
87 / Grab	app-technical	Describes managed transactional email delivery and deliverability dashboards. Monitor verification/password recovery/receipt delivery rather than only API responses.	Grab Powers Customer Engagement Across Southeast Asia with Amazon SES | Grab Case Study | AWS Published: Not determined Reviewed: 2026-10-03
88 / Getir	app-technical	Describes managed warehouse and forecasting separated from application growth needs. Separate analytics and recommendations from transactional production tables.	Getir Halves Time to Insight and Boosts Forecast Accuracy 10% Using Amazon Redshift and Amazon Forecast Published: Not determined Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
89 / Booking.com	app-technical	Describes search-ranking model serving at low latency and high request volume. Define model-serving latency/quality SLOs for learning recommendations.	Innovating with ML at scale using Amazon EKS with Booking.com | Case Study | AWS Published: Not determined Reviewed: 2026-10-03
90 / Expedia	company-technical	Expedia Group describes governed database provisioning across its portfolio. Shared-company evidence; keep database configuration and access governance repeatable.	How Expedia Group built Database as a Service (DBaaS) offering using AWS Service Catalog | AWS Cloud Operations Blog Published: 2022-04-19 Reviewed: 2026-10-03

Cohort entries 91-100

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
91 / Skyscanner	app-technical	Describes global CDN consolidation measured cache-hit rate and cost savings. Measure cache-hit rate bytes cost and regional latency for lesson media.	Delivering Travel Deals across 110 Markets Using Amazon CloudFront with Skyscanner Published: Not determined Reviewed: 2026-10-03
92 / Traveloka	app-technical	Describes API developer portal and service contract discovery. Document backend ownership API contracts and service dependencies.	How Traveloka Uses Backstage as an API Developer Portal for Amazon API Gateway | AWS Open Source Blog Published: 2023-05-31 Reviewed: 2026-10-03
93 / PayPal	app-API	Documents webhook signature validation and integration security. Verify payment-event signatures before entitlement mutation.	PayPal security guidelines for developers | PayPal Developer Published: Not determined Reviewed: 2026-10-03
94 / Wise	app-API	Documents bounded retry safe replay and reuse of idempotency identifiers. Retry state-changing requests only with stable idempotency and reconciliation.	Wise API errors: handling and recovery | Wise Platform Published: Not determined Reviewed: 2026-10-03
95 / Revolut	app-technical	Describes elastic data infrastructure and governance. Govern analytical access and scale independently from transactional storage.	Revolut case study | Google Cloud Published: Not determined Reviewed: 2026-10-03
96 / Monzo	app-technical	Describes intentionally smaller independent fallback for critical payments. Define degraded core journeys and exercise failover rather than relying on backup existence.	Monzo Bank Success Story Published: Not determined Reviewed: 2026-10-03
97 / Nubank	company-technical	Describes managed Oracle migration for brokerage-related applications and governance consolidation. Subset/company evidence; measure query latency and validate backups in context.	Nubank migrates 100+ apps to Amazon RDS for Oracle for up to 50% lower latency | Case Study | AWS Published: Not determined Reviewed: 2026-10-03

# / App or service	Disclosure scope	Disclosed practice / relevance	Primary source
98 / Robinhood	company-technical	Describes AI assistance for financial-crime investigations using governed data. Keep sensitive model context controlled and require evaluated outputs.	Robinhood Transforms Financial Crimes Investigations Using Amazon Bedrock Published: Not determined Reviewed: 2026-10-03
99 / Coinbase	subsystem-technical	Describes low-latency International Exchange and disaster recovery needs. Subsystem evidence; measure consistent latency and recoverable durable state.	Coinbase Launches an Ultra-Low-Latency Cryptocurrency Exchange on AWS | AWS Published: Not determined Reviewed: 2026-10-03
100 / Cash App	app-technical	Describes managed Kubernetes resilience and controlled fault experiments. Test dependency failure and recovery paths before incidents.	Improving platform resilience at Cash App | AWS Architecture Blog Published: 2025-05-29 Reviewed: 2026-10-03